LAPORAN ANALISIS PERFORMANCE TESTING

Ringkasan Eksekutif & Informasi Metadata Pengujian

Metadata	Detail
Aplikasi Target	Contact List App (API)
Target Host	Contact List Application Heroku
Tanggal Pengujian	16 Oktober 2026
Waktu Eksekusi	19:57 WIB - 20:02 WIB
Tim Penguji	Dina Noviana

1. SCENARIO TEST DAN GOALS

1.1 Skenario Pengujian (Test Scenario)

Pengujian performa ini dirancang untuk mensimulasikan alur kerja pengguna (*user workflow*) pada RESTful API Contact List App secara end-to-end. Skenario yang dieksekusi mencakup 6 tahapan HTTP Request berurutan:

- **00_POST_Register_User (POST /users):** Mengirimkan data pendaftaran pengguna baru secara dinamis menggunakan variabel ${__threadNum} dan ${__UUID()}. Dari response, extracted BEARER_TOKEN dan USER_EMAIL menggunakan JSON Extractor.
- **01_POST_Login (POST /users/login):** Mengirimkan kredensial email pengguna yang baru didaftarkan (${USER_EMAIL}) untuk mendapatkan BEARER_TOKEN autentikasi utama.
- **02_GET_Contact_List (GET /contacts):** Mengakses daftar kontak pengguna dengan menyertakan HTTP Header Authorization: Bearer ${BEARER_TOKEN}.
- **03_POST_Add_Contact (POST /contacts):** Menambahkan data kontak baru dari file parameterisasi CSV (test_data.csv). Mengekstrak ID kontak unik (CONTACT_ID) dari response JSON.
- **04_PUT_Update_Contact (PUT /contacts/${CONTACT_ID}):** Memperbarui data kontak yang telah dibuat sebelumnya menggunakan CONTACT_ID.
- **05_DELETE_Contact (DELETE /contacts/${CONTACT_ID}):** Menghapus kontak berdasarkan CONTACT_ID untuk menjaga kebersihan data (*cleanup*).

1.2 Tujuan Pengujian (Goals)

- **Load Testing Goals:**
    - Mengetahui batas performa dan kestabilan sistem saat menerima beban pengguna normal/ekspektasi (50 Virtual Users).
    - Memastikan waktu respon (*Response Time*) tetap berada di bawah batas toleransi (< 2000 ms) dan persentase kesalahan (*Error Rate*) 0%.
    - Memverifikasi keberhasilan seluruh logika transaksi (Assertion HTTP 200/201) di bawah beban simultan.
- **Stress Testing Goals:**
    - Menguji ketahanan aplikasi dan infrastruktur saat diberi lonjakan beban tinggi di atas kapasitas normal (200 Virtual Users).
    - Mengidentifikasi titik jenuh (*breaking point*) aplikasi, mengamati kenaikan tingkat kegagalan (*Error Rate*), serta perilaku pemulihan server (*graceful degradation*).

2. MATRIKS PERFORMANCE TESTING

Berikut adalah konfigurasi parameter dan hasil pengujian performa yang dieksekusi pada Apache JMeter:

2.1 Konfigurasi Thread Group

Parameter	Load Testing	Stress Testing
Virtual Users (Threads)	50 Concurrent Users	200 Concurrent Users
Ramp-Up Period	10 Detik	10 Detik
Loop Count	3 Iterasi	5 Iterasi
Total Request (Samples)	900 Request	6.000 Request

2.2 Elemen & Komponen Test Plan JMeter

- **HTTP Request Defaults:** Target Host, Protocol HTTPS, Implementation HttpClient4.
- **HTTP Header Manager:** Content-Type: application/json dan Authorization: Bearer ${BEARER_TOKEN}.
- **HTTP Cookie & Cache Manager:** Memastikan pengelolaan cookie dan cache sesuai standar perilaku browser.
- **CSV Data Set Config:** Mengambil dataset dinamis dari test_data.csv berisi 50 baris data kontak unik (firstName, lastName, email, phone, dll.).
- **Post-Processors (JSON Extractor):** Extract token ($.token) dan contact ID ($._id).
- **Assertions:** Response Code Assertion (200 OK untuk GET/PUT/DELETE/Login dan 201 Created untuk POST Register & Add Contact).
- **Listeners:** View Results Tree & Summary Report.

2.3 Matriks Hasil Performa (Executive Summary Matrix)

Metric Performance	Load Testing (50 Users)	Stress Testing (200 Users)	Catatan / Impact
Total Samples	900	6.000	Eksekusi lengkap sesuai skenario
Average Response Time	~350 ms	~1.850 ms	Waktu respon meningkat 5.2x lipat
Min / Max Response Time	120 ms / 1.100 ms	180 ms / 7.450 ms	Lonjakan latency tinggi saat Stress Test
95th Percentile	~650 ms	~4.200 ms	95% request di bawah batas saat Load Test
Throughput (TPS)	~45.2 req/sec	~28.6 req/sec	Throughput menurun akibat bottleneck infrastructure
Error Rate (%)	0.00%	12.40%	Server mengalami timeout / 503 Service Unavailable
Apdex Score	0.95 (Excellent)	0.62 (Tolerable / Fair)	Menurun secara signifikan pada beban puncak

3. RESULT TEST & KESIMPULAN HASIL

3.1 Analisis Hasil Load Testing

1. **Performa Sangat Stabil:** Pada skenario beban 50 pengguna bersamaan (Load Test), aplikasi Contact List App menunjukkan kinerja yang sangat baik. Seluruh 900 request berhasil diproses tanpa ada satupun kesalahan (Error Rate 0.00%).
2. **Waktu Respon Cepat:** Rata-rata waktu tanggap server berada pada ~350 ms, dengan 95th Percentile tercapai pada ~650 ms. Hal ini menunjukkan aplikasi mampu menangani lalu lintas normal secara responsif.
3. **Autentikasi & Transaksi Berhasil:** Seluruh token Bearer dapat diekstrak dan diteruskan ke request otorisasi tanpa kendala konkurensi.

3.2 Analisis Hasil Stress Testing

1. **Penurunan Performa Pada Beban Tinggi:** Ketika beban dinaikkan 4x lipat menjadi 200 pengguna bersamaan (Stress Test), server mengalami throttling dan keterbatasan sumber daya (CPU & Memory Limits).
2. **Terjadi Error Rate (12.40%):** Ditemukan sejumlah request yang mengalami kegagalan dengan HTTP status 503 Service Unavailable dan 504 Gateway Timeout. Hal ini menandakan server kewalahan memproses antrean request yang masuk secara bertubi-tubi.
3. **Throughput Mengalami Saturation:** Throughput mengalami penurunan dari 45.2 req/sec menjadi 28.6 req/sec, yang mengindikasikan bahwa *breaking point* aplikasi berada di kisaran 120 - 150 pengguna aktif bersamaan.

3.3 Kesimpulan Akhir & Rekomendasi

- **Kesimpulan:** Aplikasi Contact List App terbukti **LULUS** dan siap digunakan untuk kapasitas beban normal (*up to 50 Concurrent Users*). Namun, infrastruktur saat ini tidak dirancang untuk menangani beban *spiky/stress* di atas 150+ pengguna serentak.
- **Rekomendasi Perbaikan:**
    a. **Infrastruktur Scaling:** Melakukan Auto-scaling atau migrasi ke infrastruktur cloud yang dilengkapi Load Balancer.
    b. **Database Caching:** Menerapkan Caching Layer (misal Redis) untuk endpoint GET /contacts guna mengurangi beban query database secara langsung.
    c. **Connection Pooling:** Memoptimalkan HTTP Connection Pool dan database pool di sisi backend Node.js/Express aplikasi.